Model Question Paper – Cloud Data Engineer - Jain University

NAME : NUSUM YELLESWAR REDDY

HCL ID : HCLTFP2260249

USN : 23BTRCT092

DEPATRMENT : CSE

SECTION A: ADVANCED SQL

Question 1: Database Design and Advanced Queries

```
46    select * from Departments ;
47
48
49
50
```

Output | Result #7

DepartmentID	DepartmentName	Location
101	HR	Bangalore
102	Engineering	Hyderabad
103	Finance	Mumbai
104	Sales	Delhi
105	R&D	Bangalore

```
46    select * from  Employees;
47
48
49
```

Output | Result #7

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID
1	yelleswar	101	75000.00	01/10/2022	NULL
2	anish	102	120000.00	03/15/2021	1
3	pranavi	102	95000.00	06/01/2023	2
4	vishnu	103	80000.00	11/20/2022	1
5	hemanth	102	100000.00	07/12/2020	2
6	pavan	104	60000.00	01/05/2023	1
7	palani	104	65000.00	02/18/2023	6
8	koushik	101	70000.00	05/30/2022	1
9	tejash	103	85000.00	09/10/2021	4
10	aravind	102	130000.00	04/22/2019	2

CHEMAS

a. Display employee names, department names, and manager names using joins.

```
60
61  SELECT e.EmployeeName, d.DepartmentName, m.EmployeeName AS ManagerName
62  FROM Employees e
63  LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
64  LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Output | Result #8

EmployeeName	DepartmentName	ManagerName
pavan	Sales	yelleswar
palani	Sales	pavan
koushik	HR	yelleswar
tejash	Finance	vishnu
aravind	Engineering	anish
rohith	Sales	pavan
naveen	HR	yelleswar
sanjay	Engineering	anish
mukund	Finance	vishnu
Ravi	Sales	pavan

b. Find employees earning more than their department's average salary.

```
66
67  SELECT e.EmployeeName, e.Salary, e.DepartmentID
68  FROM Employees e
69  WHERE e.Salary > (SELECT AVG(Salary) FROM Employees WHERE DepartmentID = e.DepartmentID);
```

Output | Result #9

EmployeeName	Salary	DepartmentID
yelleswar	75000.00	101
anish	120000.00	102
pavan	60000.00	104
palani	65000.00	104
tejash	85000.00	103
aravind	130000.00	102
Ravi	62000.00	104

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

```
71
72  SELECT MAX(Salary) AS SecondHighestSalary
73  FROM Employees
74  WHERE Salary < (SELECT MAX(Salary) FROM Employees);
75
```

Output | Result #10

SecondHighestSalary
120000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

```
76
77  SELECT * FROM (
78      SELECT EmployeeName, DepartmentID, Salary,
79      DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS rnk
80      FROM Employees
81  ) t WHERE rnk <= 3;
```

Output | Result #11

EmployeeName	DepartmentID	Salary	rnk
naveen	101	70000.00	2
aravind	102	130000.00	1
anish	102	120000.00	2
hemanth	102	100000.00	3
tejash	103	85000.00	1
vishnu	103	80000.00	2
mukund	103	78000.00	3
palani	104	65000.00	1
Ravi	104	62000.00	2
pavan	104	60000.00	3

Question 2: Sales Analytics and Query Optimization

```
9   SELECT * FROM Sales;
10
11  INSERT INTO  Sales VALUES
```

Output | Result #2

SaleID	CustomerID	SaleDate	ProductName	Quantity	UnitPrice
1	1	10/23/2026	laptop	2	80000.00
2	2	02/18/2026	smartphone	5	40000.00
3	5	01/23/2025	tv	1	80000.00
4	4	06/20/2026	laptop	3	30000.00
5	2	04/19/2024	watches	4	5000.00
6	3	01/06/2022	samrtphones	1	80000.00
7	1	09/02/2023	watches	4	2000.00
8	4	07/17/2026	headphones	4	1800.00
9	2	02/28/2024	samrtphones	1	45000.00
10	5	08/20/2023	laptop	1	55000.00

a. Calculate total revenue for each product.

```
28  SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue
29  FROM Sales
30  GROUP BY ProductName;
31
```

Output | Result #4

ProductName	TotalRevenue
laptop	385000.00
smartphone	200000.00
tv	80000.00
watches	46100.00
samrtphones	125000.00
headphones	15200.00
smartphones	30000.00

b. Calculate monthly revenue and month-over-month revenue change using a window function.

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.

Query :
SELECT * FROM Employees
WHERE JoiningDate >= '2025-01-01' AND JoiningDate < '2026-01-01';

Question 3: SQL Troubleshooting

a. Explain how an execution plan helps identify expensive operations in a query.

ANS : Execution Plan: EXPLAIN / EXPLAIN ANALYZE shows access type, index usage, row estimates, cost, join order. Look for Seq Scan, high cost sorts, nested loops on large tables to find expensive ops.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

ANS : Index Strategy: Composite index CREATE INDEX idx_emp_dept_salary ON Employees(DepartmentID, Salary); DepartmentID first for equality filter, Salary second for range/sort. Include covering if needed.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

ANS :

SELECT * FROM Employees
WHERE JoiningDate >= '2025-01-01' AND JoiningDate < '2026-01-01';

SECTION B: LINUX

Question 4: Log Analysis

a. Find all lines containing ERROR.

ANS : grep “ERROR” /var/log/application.log

b. Display the 20 most recent error entries in the file.

ANS : grep “ERROR” /var/log/application.log | tail -n 20

c. Count the total number of error entries.

ANS : grep -c “ERROR” /var/log/application.log

d. Extract error entries for a specified date into a separate file.

ANS :grep “^2026-10-28. “ERROR” /var/log/application.log
>errors_2026-10-28.log

e. Explain how you would calculate the number of errors occurring in each hour.

ANS : grep”^2026-10-28.last_hour.ERROR”
/var/log/application.log> error_last_hour.log

Question 5: Permissions and System Troubleshooting

a. Create a group named finance and a report file, assuming you have the required privileges.

ANS : sudo group add finance

Touch report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access.

ANS : sudo chown “user”: finance report.txt

c. Identify the processes consuming the most CPU.

ANS : top

d. Display memory usage and filesystem disk utilization.

ANS : free-h

df-h

E. Find files larger than 100 MB under a specified application directory.

ANS : find /path/to/application-type f-size +100M

SECTION C: PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder.appName("EmployeeDataFrame").getOrCreate()

data = [
    (101, "Ravi", "IT", 75000, "2023-01-15"),
    (102, "Sita", "HR", 45000, "2022-03-10"),
    (103, "Arjun", "IT", 85000, "2021-07-20"),
    (104, "Priya", "Finance", 65000, "2023-05-12"),
    (105, "Kiran", "HR", 55000, "2022-11-01"),
    (106, "Meena", "IT", 95000, "2020-09-18"),
    (107, "Rahul", "Finance", 72000, "2021-02-25"),
    (108, "Anu", "IT", 60000, "2024-01-08"),
    (109, "Vijay", "HR", 48000, "2023-08-14"),
    (110, "Sneha", "Finance", 80000, "2020-06-30"),
    (111, "Manoj", "IT", 68000, "2022-12-05"),
    (112, "Divya", "HR", 62000, "2021-04-17"),
    (113, "Suresh", "Finance", 58000, "2024-02-11"),
    (114, "Lakshmi", "IT", 90000, "2019-10-22"),
    (115, "Naveen", "HR", 70000, "2020-12-19")
]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]
```

```
df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

# a. Display schema and first five records
print("A. SCHEMA")
df.printSchema()

print("FIRST FIVE RECORDS")
df.show(5, truncate=False)

# b. Filter salary greater than 60000
print("B. SALARY GREATER THAN 60000")
df.filter(col("Salary") > 60000).show(truncate=False)

# c. Calculate annual salary
print("C. ANNUAL SALARY")
df.withColumn(
   "AnnualSalary", col("Salary") * 12
).show(truncate=False)

# d. Average salary and employee count by department
print("D. DEPARTMENT SUMMARY")
df.groupBy("Department").agg(
   avg("Salary").alias("AverageSalary"),
   count("*").alias("EmployeeCount")
).show(truncate=False)

# e. Five highest-paid employees
```

print("E. TOP FIVE HIGHEST-PAID EMPLOYEES")

df.orderBy(col("Salary").desc()).show(5, truncate=False)

spark.stop()

a. Display the schema and first five records.

```
A. SCHEMA
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

FIRST FIVE RECORDS
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Ravi        |IT        |75000 |2023-01-15 |
|102       |Sita        |HR        |45000 |2022-03-10 |
|103       |Arjun       |IT        |85000 |2021-07-20 |
|104       |Priya       |Finance   |65000 |2023-05-12 |
|105       |Kiran       |HR        |55000 |2022-11-01 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000.

```
B. SALARY GREATER THAN 60000
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Ravi        |IT        |75000 |2023-01-15 |
|103       |Arjun       |IT        |85000 |2021-07-20 |
|104       |Priya       |Finance   |65000 |2023-05-12 |
|106       |Meena       |IT        |95000 |2020-09-18 |
|107       |Rahul       |Finance   |72000 |2021-02-25 |
|110       |Sneha       |Finance   |80000 |2020-06-30 |
|111       |Manoj       |IT        |68000 |2022-12-05 |
|112       |Divya       |HR        |62000 |2021-04-17 |
|114       |Lakshmi     |IT        |90000 |2019-10-22 |
|115       |Naveen      |HR        |70000 |2020-12-19 |
```

c. Add an AnnualSalary column calculated from the monthly salary.

```
C. ANNUAL SALARY
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|101       |Ravi        |IT        |75000 |2023-01-15 |900000      |
|102       |Sita        |HR        |45000 |2022-03-10 |540000      |
|103       |Arjun       |IT        |85000 |2021-07-20 |1020000     |
|104       |Priya       |Finance   |65000 |2023-05-12 |780000      |
|105       |Kiran       |HR        |55000 |2022-11-01 |660000      |
|106       |Meena       |IT        |95000 |2020-09-18 |1140000     |
|107       |Rahul       |Finance   |72000 |2021-02-25 |864000      |
|108       |Anu         |IT        |60000 |2024-01-08 |720000      |
|109       |Vijay       |HR        |48000 |2023-08-14 |576000      |
|110       |Sneha       |Finance   |80000 |2020-06-30 |960000      |
|111       |Manoj       |IT        |68000 |2022-12-05 |816000      |
|112       |Divya       |HR        |62000 |2021-04-17 |744000      |
|113       |Suresh      |Finance   |58000 |2024-02-11 |696000      |
|114       |Lakshmi     |IT        |90000 |2019-10-22 |1080000     |
|115       |Naveen      |HR        |70000 |2020-12-19 |840000      |
+----------+------------+----------+------+-----------+------------+
```

d. Group employees by department and calculate the average salary and employee count.

e. Display the five highest-paid employees.

```
D. DEPARTMENT SUMMARY
+----------+-----------------+-------------+
|Department|AverageSalary    |EmployeeCount|
+----------+-----------------+-------------+
|IT        |78833.33333333333|6            |
|HR        |56000.0          |5            |
|Finance   |68750.0          |4            |
+----------+-----------------+-------------+

E. TOP FIVE HIGHEST-PAID EMPLOYEES
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|106       |Meena       |IT        |95000 |2020-09-18 |
|114       |Lakshmi     |IT        |90000 |2019-10-22 |
|103       |Arjun       |IT        |85000 |2021-07-20 |
|110       |Sneha       |Finance   |80000 |2020-06-30 |
|101       |Ravi        |IT        |75000 |2023-01-15 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation

```
[4]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, sum
from pyspark.sql.functions import expr


spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

df = spark.read.csv("transactions.csv", header=True, inferSchema=True)

print("Original Data:")
df.show()

# Remove duplicates
df = df.dropDuplicates(["TransactionID"])


df = df.withColumn(
    "TransactionDate",
    expr("try_to_date(TransactionDate, 'yyyy-MM-dd')")
)
```

```
df = df.filter(
    col("CustomerID").isNotNull() &
    col("TransactionDate").isNotNull() &
    col("Quantity").isNotNull() &
    col("UnitPrice").isNotNull()
)

df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("Cleaned Data:")
df.show()

print("Revenue by Product:")
df.groupBy("Product").agg(
    sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show()
```

```
Original Data:
+-------------------+----------+---------------+--------+--------+---------+
|      TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------------+----------+---------------+--------+--------+---------+
|2026-10-28 01:00:00|        C1|     2026-01-10|  Laptop|       1|    50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 03:00:00|        C3|        invalid|Keyboard|       1|     1500|
|2026-10-28 04:00:00|      NULL|     2026-01-12|  Laptop|       1|    50000|
+-------------------+----------+---------------+--------+--------+---------+

Cleaned Data:
+-------------------+----------+---------------+-------+--------+---------+-------+
|      TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------------+----------+---------------+-------+--------+---------+-------+
|2026-10-28 01:00:00|        C1|     2026-01-10| Laptop|       1|    50000|  50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|  Mouse|       2|      500|   1000|
+-------------------+----------+---------------+-------+--------+---------+-------+

Revenue by Product:
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
| Laptop|       50000|
|  Mouse|        1000|
+-------+------------+
```

Question 9: PySpark Joins and Performance

```
transcations > customers.py > ...
1
2   import os
3   import sys
4
5   os.environ["PYSPARK_PYTHON"] = sys.executable
6   os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
7
8   from pyspark.sql import SparkSession
9   from pyspark.sql.functions import col, sum as _sum
10
11  # Create Spark session
12  spark = SparkSession.builder \
13      .appName("CustomerOrders") \
14      .master("local[*]") \
15      .getOrCreate()
```

a. Load both datasets into DataFrames.

```
+----------+------------+----------+
|CustomerID|CustomerName|      City|
+----------+------------+----------+
|       101|        Ravi| Hyderabad|
|       102|        Sita|   Chennai|
|       103|       Arjun| Bangalore|
|       104|       Priya| Hyderabad|
|       105|       Kiran|Vijayawada|
|       106|       Meena|   Chennai|
+----------+------------+----------+

ORDERS
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|   1001|       101|       2500|2026-10-20|
|   1002|       102|       1800|2026-10-21|
|   1003|       101|       3200|2026-10-22|
|   1004|       103|       4500|2026-10-23|
|   1005|       104|       1500|2026-10-24|
|   1006|       102|       2200|2026-10-25|
```

b. Join the datasets to display orders with customer details.

```
ORDERS WITH CUSTOMER DETAILS
+----------+------------+----------+-------+-----------+----------+
|CustomerID|CustomerName|      City|OrderID|OrderAmount| OrderDate|
+----------+------------+----------+-------+-----------+----------+
|       101|        Ravi| Hyderabad|   1001|       2500|2026-10-20|
|       102|        Sita|   Chennai|   1002|       1800|2026-10-21|
|       101|        Ravi| Hyderabad|   1003|       3200|2026-10-22|
|       103|       Arjun| Bangalore|   1004|       4500|2026-10-23|
|       104|       Priya| Hyderabad|   1005|       1500|2026-10-24|
|       102|        Sita|   Chennai|   1006|       2200|2026-10-25|
|       105|       Kiran|Vijayawada|   1007|       3000|2026-10-26|
+----------+------------+----------+-------+-----------+----------+
```

c. Identify customers who have never placed an order.

```
45  customer_orders.show()
46
47  # c. Customers who never placed an order
48  print("CUSTOMERS WITHOUT ORDERS")
49
50  customers_without_orders = customers.join(
51      orders,
52      on="CustomerID",
53      how="left_anti"
54  )
55
56  customers_without_orders.show()
```

Problems Output Debug Console **Terminal** Ports

```
(.venv) PS C:\Users\User\OneDrive\Desktop\pyspark\transcations> python -u customers.py
|       105|       Kiran|Vijayawada|   1007|       3000|2026-10-26|
+----------+------------+----------+-------+-----------+----------+

CUSTOMERS WITHOUT ORDERS
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|       106|       Meena|Chennai|
```

d. Calculate the total order value by city.

```
(.venv) PS C:\Users\User\OneDrive\Desktop\pyspark\transcations> python -u customers.py
CUSTOMERS WITHOUT ORDERS
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|       106|       Meena|Chennai|
+----------+------------+-------+
```

e. Explain when a broadcast join is useful and what conditions make it appropriate.

```
(.venv) PS C:\Users\User\OneDrive\Desktop\pyspark\transcations> python -
+----------+---------------+
|      City|TotalOrderValue|
+----------+---------------+
| Hyderabad|           7200|
| Bangalore|           4500|
|   Chennai|           4000|
|Vijayawada|           3000|
+----------+---------------+

(.venv) PS C:\Users\User\OneDrive\Desktop\pyspark\transcations> SUCCESS:
```

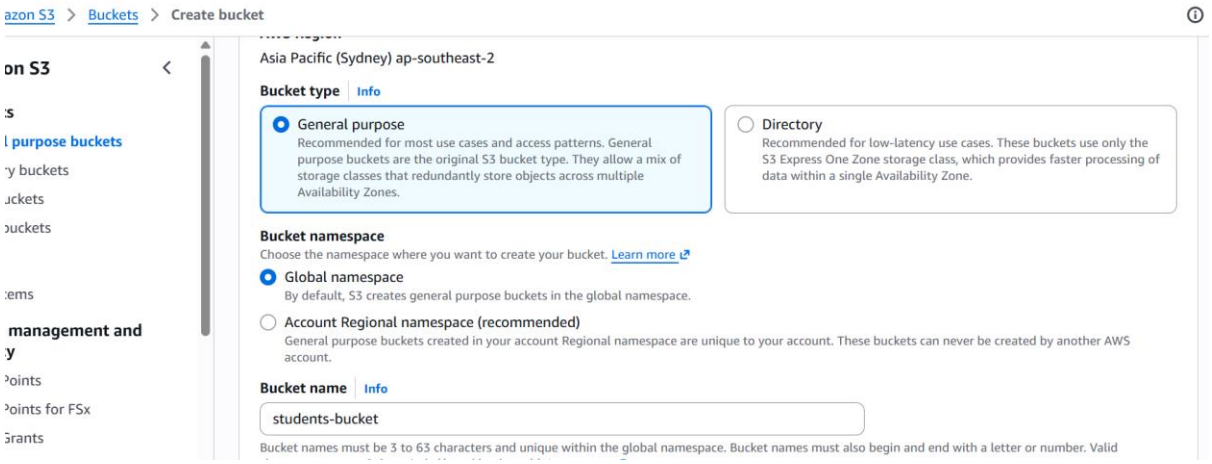
f. Transformations vs. Actions (1 Mark)

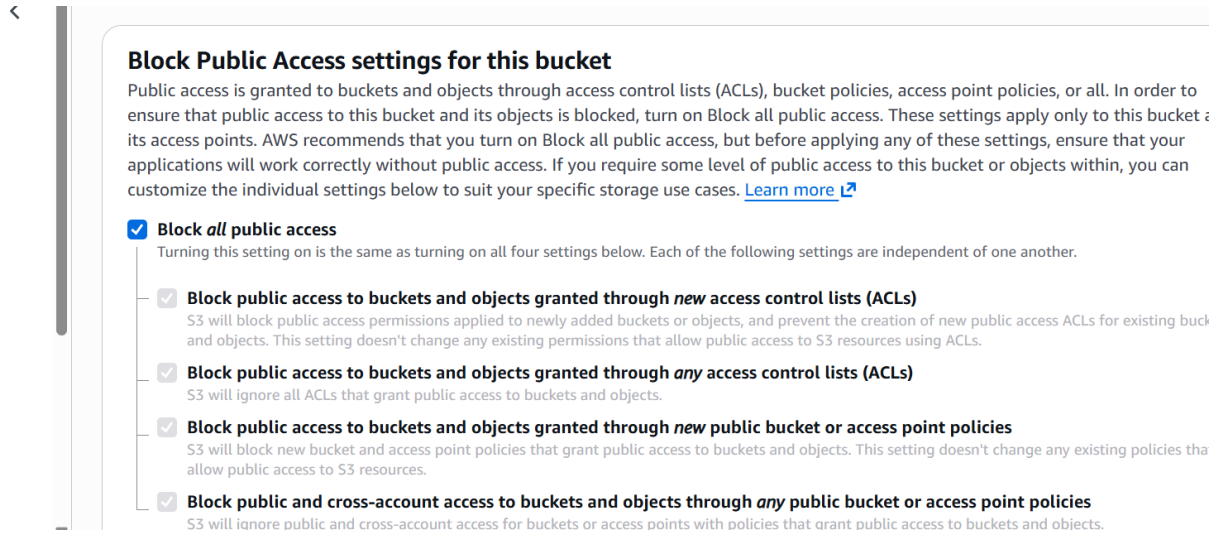
- Transformations: Create a new DataFrame or execution plan. Example: filter().
- Actions: Execute computations and return results. Example: show().

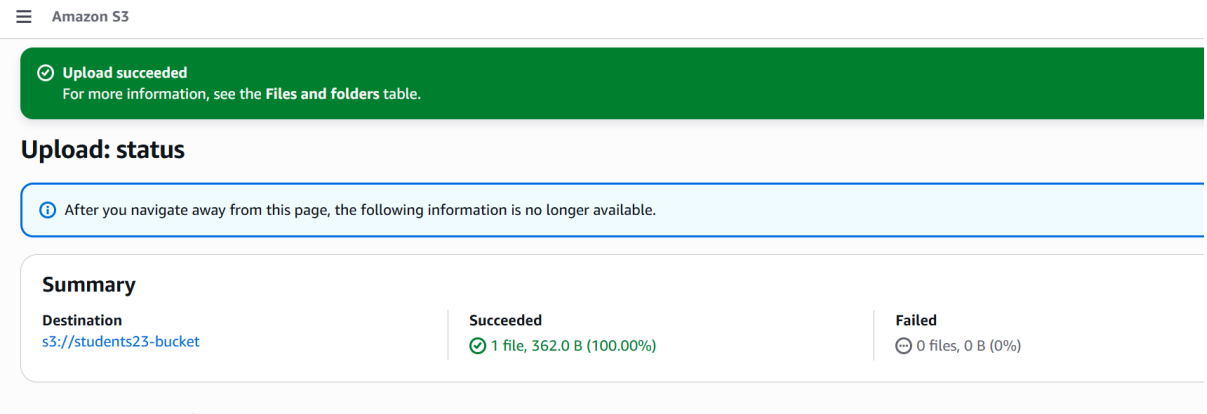
SECTION D: AWS CLOUD SERVICES

Question 10: S3 and IAM Security

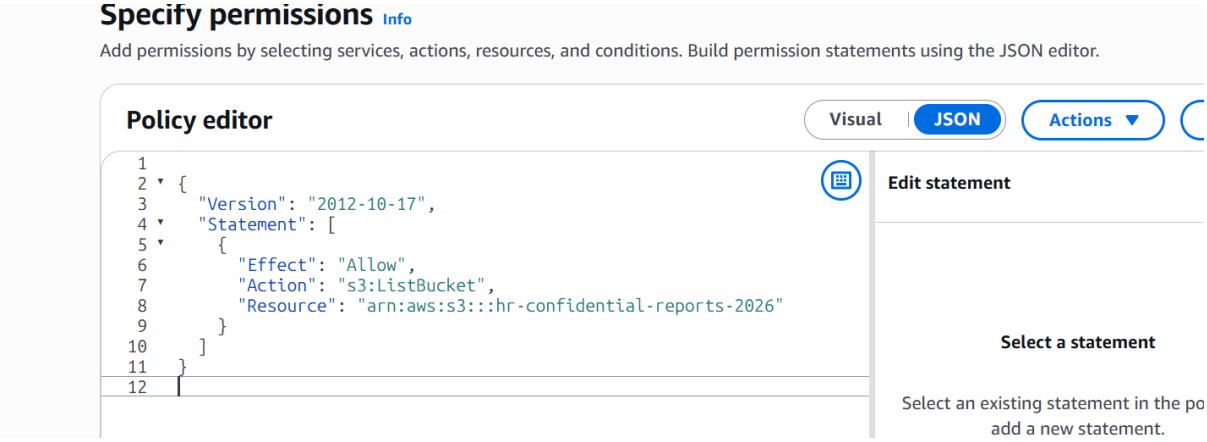
a. Create an S3 bucket with Block Public Access enabled and upload a sample file.







b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.



c. Explain how you would verify that an unauthorized user cannot read the file.

answer: If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data. (1 mark)

answer: Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

e. Explain how execution history helps troubleshoot a failed workflow.

An execution history acts as a detailed, chronological ledger of everything that happened during a workflow run, making it the most critical tool for diagnosing and fixing failures.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.

Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity.

Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization.

Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

Question 12: Event-Driven Processing

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code.

AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

b. Explain how an S3 upload can trigger processing.

Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

c. Explain the IAM permissions needed to read the input file and write the results.

Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

d. Explain how errors and processing logs can be captured.

Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

e. Explain how to prevent duplicate processing or recursive triggering.

Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification.

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

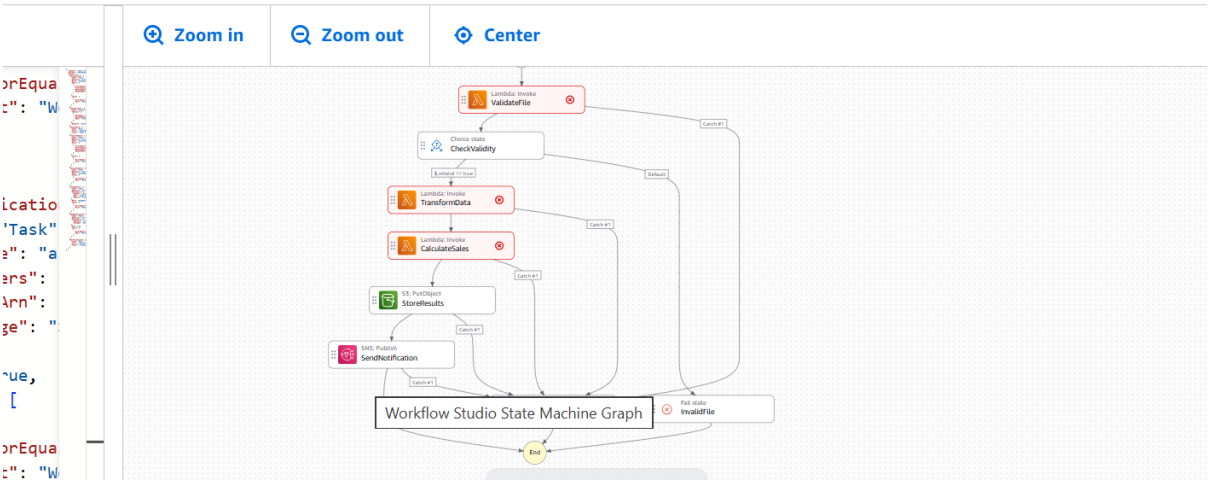
Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

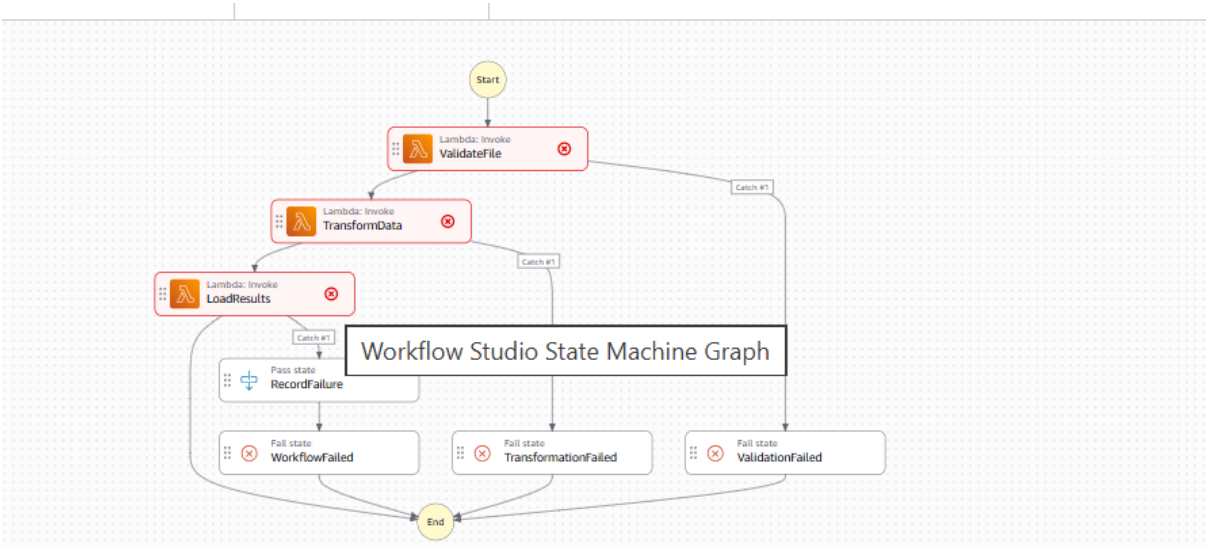
e. Explain how execution history helps troubleshoot a failed workflow.

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.



Question 14: Step Functions Implementation

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.



b. Configure a retry policy for the transformation task. (2 marks)

The Retry state retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.

c. Configure a Catch transition for the loading task. (1 mark)

The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.

d. Explain how data from one task is passed to the next task. (1 mark)

Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.

e. Explain how you would test successful and failed executions. (1 mark)

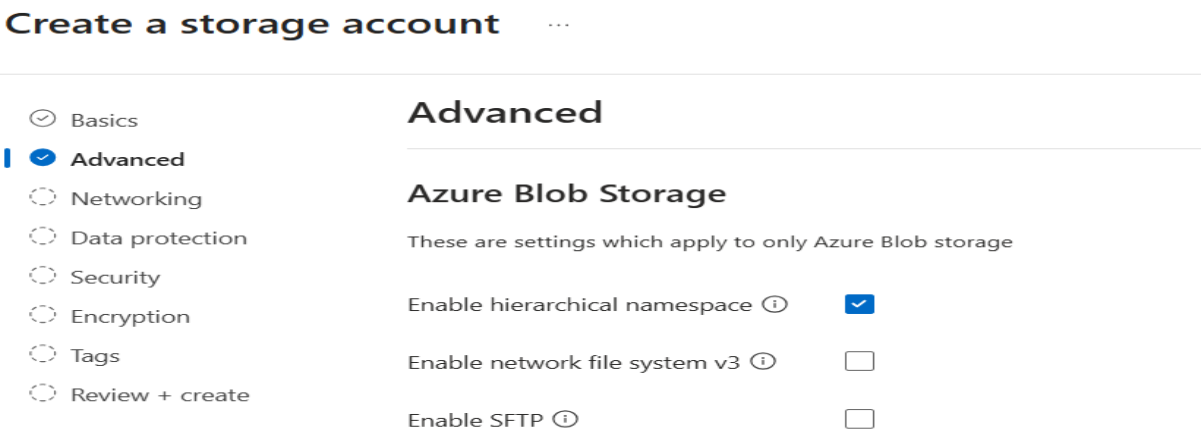
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

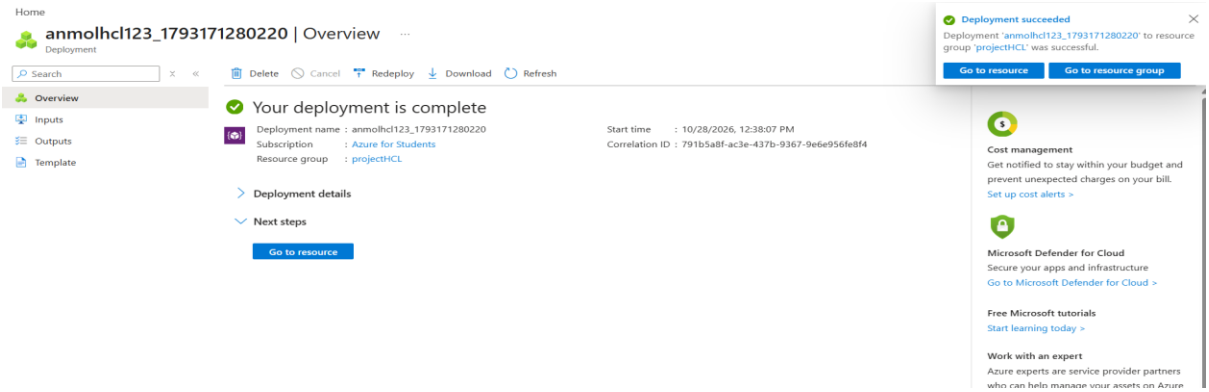
SECTION F: MICROSOFT AZURE || Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security

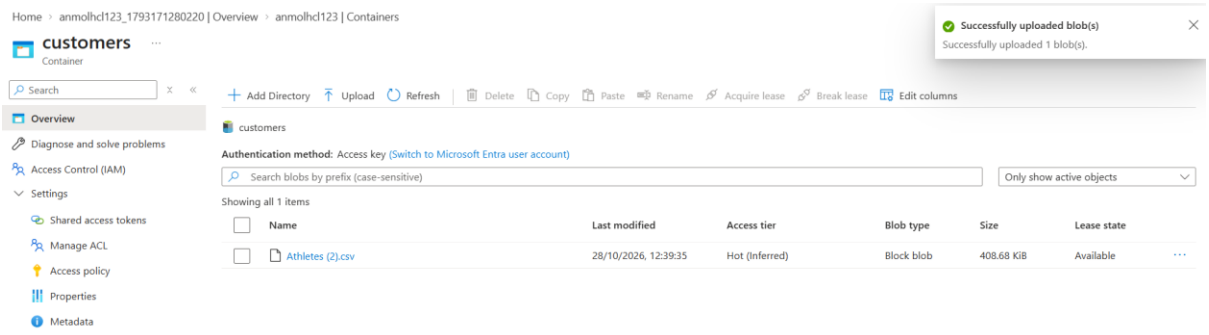
a. Configure a storage account with hierarchical namespace enabled.

Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset.



Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

c. Explain how Microsoft Entra ID and Azure RBAC manage access to the data. (2 marks)

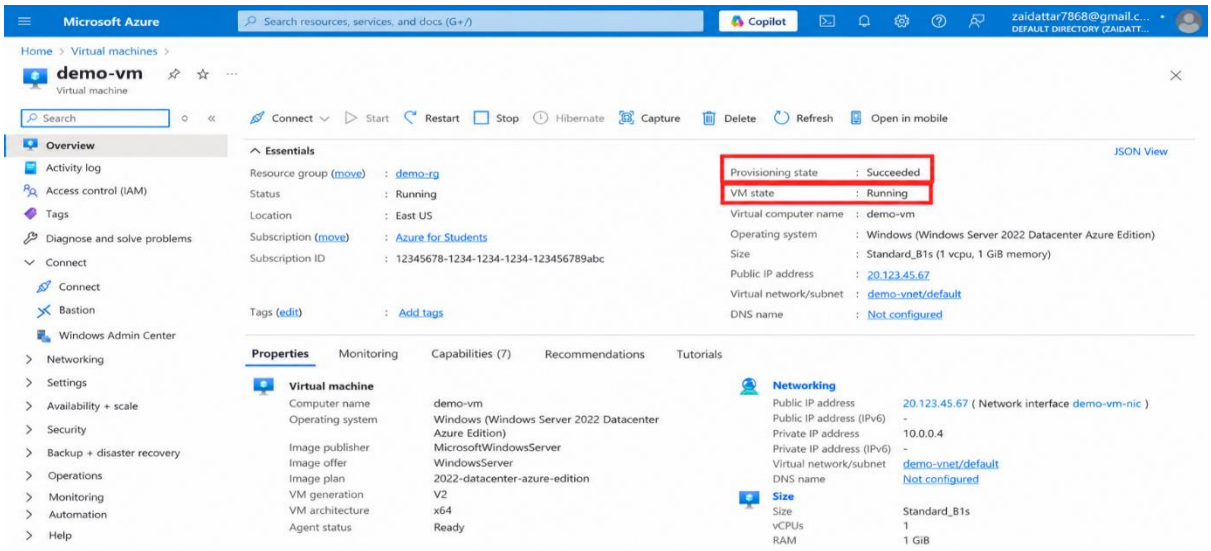
Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured

Question 16: Azure VM Troubleshooting (5 Marks)

a. Verify the VM's provisioning and power states.



b. How can NSG rules prevent connectivity?

- Network Security Group (NSG) rules control inbound and outbound traffic.
- If inbound rules block ports such as 22 (SSH) or 3389 (RDP), administrators cannot connect to the VM.

c. How do subnet and routing configurations affect connectivity?

- Incorrect subnet configuration or route table entries can block traffic to the VM.
- Verify the subnet, route table, and network connectivity settings.

d**. Recommend a secure troubleshooting method.**

- Use Azure Bastion to securely connect to the VM through the Azure Portal without exposing its public IP address.
- Alternatively, use Just-in-Time (JIT) VM access to open required ports only when needed, reducing exposure to the public internet.

Question 17: Azure Data Integration

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement.

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Describe how to organize files by ingestion date. (1 mark)

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Explain how to handle malformed records and missing values. (1 mark)

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.